

Documentation Technique — API Django REST (Backend)

Projet PokéCard Masters — BTS SIO option SLAM

1. Présentation du projet

PokéCard Masters est une application de collection de cartes Pokémon TCG (Trading Card Game). Le backend constitue le cœur du système : il expose une **API RESTful** construite avec **Django** et **Django Ninja**, permet la gestion des utilisateurs, l'authentification (Google OAuth2 + Email/Mot de passe), la consultation du catalogue de cartes, et la simulation d'ouverture de boosters.

Les données de cartes sont importées depuis l'API publique **TCGDex**, stockées dans une base de données **MySQL**, puis servies aux clients mobiles via des endpoints JSON sécurisés par **JWT (JSON Web Token)**.

2. Cahier des charges

2.1 Contexte

Dans le cadre du BTS SIO option SLAM, ce projet répond à un besoin fictif mais réaliste : proposer une application mobile permettant à des collectionneurs de cartes Pokémon de gérer leur collection numérique, d'ouvrir des boosters virtuels et de consulter le catalogue complet.

2.2 Besoins fonctionnels

ID	Besoin	Priorité
F01	Inscription d'un utilisateur (email + mot de passe)	Haute
F02	Connexion via email/mot de passe	Haute
F03	Connexion via Google OAuth2	Haute

ID	Besoin	Priorité
F04	Consultation du catalogue de cartes	Haute
F05	Consultation de la collection personnelle	Haute
F06	Simulation d'ouverture de booster (10 cartes)	Moyenne
F07	Pagination des résultats	Moyenne
F08	Filtrage des cartes par type	Basse
F09	Import/export de la collection utilisateur	Basse

2.3 Besoins non fonctionnels

- **Sécurité** : toutes les routes sensibles sont protégées par JWT
- **Performance** : utilisation de `select_related` pour limiter les requêtes N+1
- **Fiabilité** : opérations critiques (ouverture de booster) gérées via des transactions atomiques
- **Maintenabilité** : architecture modulaire Django, code typé avec Pydantic (Schemas Django Ninja)

3. Stack technique

Composant	Technologie	Version
Langage	Python	3.13
Framework web	Django	—
Framework API	Django Ninja	—
Base de données	MySQL	—
Authentification	JWT (PyJWT) + Google OAuth2	—
Gestionnaire de paquets	uv	—
Source des données cartes	TCGDex API	—

4. Architecture du projet

```
project/
├── app/
│   ├── models.py           # Modèles de données (ORM Django)
│   ├── api.py              # Endpoints REST (Django Ninja)
│   ├── authentication.py    # Logique JWT (Google + App)
│   └── ...
├── manage.py               # CLI Django
├── import_cards.py         # Script d'import TCGDex → MySQL
├── project/
│   ├── settings.py         # Configuration Django
│   └── urls.py              # Routage principal
└── pyproject.toml          # Dépendances (uv)
```

Pattern architectural : MVT + API Layer

Django suit le pattern **MVT** (Model – View – Template). Dans ce projet, la couche Template est remplacée par une **API Layer** (Django Ninja), ce qui donne :

```
Client Mobile (Expo)
    |
    ▼ HTTP + JWT
Django Ninja API ↔ Schemas (Pydantic)
    |
    ▼
Django ORM (Models)
    |
    ▼
MySQL Database
```

5. Modèles de données

5.1 Diagramme Entité-Relation (ERD)

Card		User
id (PK)		id (PK)
name		user_id
card_id		name
image	N	password (hashed)
category	N	email (unique)

Champ	Type	Description
<code>hp</code>	IntegerField(null)	Points de vie
<code>evolution</code>	CharField(20, null)	Stade d'évolution
<code>description</code>	TextField(200, null)	Description de la carte
<code>evolve_from</code>	CharField(100, null)	Pokémon pré-évolution

User — Utilisateur de l'application.

Champ	Type	Description
<code>id</code>	AutoField (PK)	Identifiant interne
<code>user_id</code>	CharField(100)	Sub Google ou <code>local_<uuid></code>
<code>name</code>	CharField(200)	Nom d'affichage
<code>password</code>	CharField(128)	Hash bcrypt (vide si Google)
<code>email</code>	EmailField (unique)	Email, clé de liaison des comptes
<code>last_booster_opened</code>	DateField(null)	Date dernier booster
<code>booster_count</code>	IntegerField	Compteur de boosters ouverts

PlayerCard — Liaison entre un utilisateur et ses cartes (table de jointure enrichie).

Champ	Type	Description
<code>card_user</code>	FK → User	Propriétaire
<code>card</code>	FK → Card	Carte possédée
<code>quantity</code>	IntegerField	Nombre d'exemplaires

6. Authentication

6.1 Double méthode d'authentification

Le système supporte deux flux d'authentification qui convergent vers un même format de token JWT.



6.2 Liaison des comptes

Un utilisateur peut posséder les deux méthodes sur le même compte. Le champ `email` est la **clé de liaison** : si un utilisateur Google tente de s'inscrire avec un email déjà existant (compte local), le mot de passe est simplement ajouté au compte existant.

6.3 Sécurité des mots de passe

Les mots de passe sont hashés avec **PBKDF2-SHA256** via `django.contrib.auth.hashers.make_password()`, jamais stockés en clair.

7. Référence des endpoints API

Base URL : `http://<host>:8000/api/`

Endpoint	Méthode	Auth JWT	Description
<code>/hello</code>	GET	Non	Test de connectivité
<code>/auth/register</code>	POST	Non	Inscription email/password
<code>/auth/login</code>	POST	Non	Connexion email/password

Endpoint	Méthode	Auth JWT	Description
<code>/me</code>	GET	Oui	Profil utilisateur (crée si 1ère connexion Google)
<code>/card</code>	GET	Oui	Liste toutes les cartes
<code>/cards</code>	GET	Non	Liste cartes avec filtre optionnel <code>?types=Feu</code>
<code>/player/card</code>	GET	Oui	Collection personnelle de l'utilisateur
<code>/booster/open</code>	POST	Oui	Ouvre un booster (10 cartes)
<code>/user/pagination</code>	GET	Non	Liste paginée des cartes

Exemple de requête — Inscription

Request

```
POST /api/auth/register
Content-Type: application/json

{
  "email": "ash@pokemon.com",
  "password": "pikachu123",
  "name": "Ash Ketchum"
}
```

Response 201

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... ",
  "user": {
    "id": 1,
    "name": "Ash Ketchum",
    "email": "ash@pokemon.com"
  }
}
```

Codes de réponse

Code	Signification
201	Compte créé avec succès
400	Champ manquant ou mot de passe < 6 caractères
409	Email déjà utilisé avec mot de passe

8. Mécanique des boosters

8.1 Règle de tirage

L'ouverture d'un booster génère **10 cartes** selon une distribution probabiliste inspirée du jeu réel :

Slot	Quantité	Pool	Détail
Communes	6	<code>common_cards</code>	One Diamond, Two Diamond
Peu Communes	2	<code>uncommon_cards</code>	Three Diamond, Four Diamond
Rares (pondérées)	2	<code>rare_cards</code>	Tirage pondéré

8.2 Poids des raretés (cartes rares)

Rareté	Poids	Probabilité relative
One Shiny	5	~35.5%
One Star	4	~28.4%
Two Star	2.5	~17.7%
Three Star	1.5	~10.6%
Two Shiny	0.8	~5.7%
Crown	0.3	~2.1%

8.3 Gestion des transactions

L'ouverture de booster utilise `transaction.atomic()` pour garantir l'**atomicité** : soit toutes les cartes sont ajoutées à la collection, soit aucune (en cas d'erreur). La mise à jour du

`booster_count` est effectuée via `F("booster_count") + 1` pour éviter les conditions de course (race conditions) en contexte concurrent.

9. Import des cartes (TCGDex)

Le script `import_cards.py` interroge l'API publique TCGDex pour alimenter la base de données :

```
TCGDex API
|
▼ HTTP GET /sets → liste des sets
|
▼ HTTP GET /cards?set=<id> → cartes d'un set
|
▼ Normalisation des données (rareté, types, HP...)
|
▼ bulk_create / update_or_create → MySQL
```

Ce script est exécuté en one-shot avant le démarrage du serveur : `uv run python project/import_cards.py`.

10. Installation et démarrage

Prérequis

- Python 3.13
- MySQL (base `pokemon`, user `root`, port 3306)
- `uv` (gestionnaire de paquets Python)

Étapes

```
# 1. Cloner le repository
git clone https://github.com/PokeCard-Masters/pokecard_masters.git
cd pokecard_masters

# 2. Installer les dépendances Python
uv sync

# 3. Configurer les variables d'environnement
cp project/.env.example project/.env
```

```
# Renseigner : GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, DJANGO_SECRET_KEY

# 4. Appliquer les migrations
uv run python project/manage.py migrate

# 5. Importer les cartes
uv run python project/import_cards.py

# 6. Lancer le serveur de développement
uv run python project/manage.py runserver
# → http://localhost:8000
```

Variables d'environnement requises

Variable	Description
<code>DJANGO_SECRET_KEY</code>	Clé secrète Django
<code>GOOGLE_CLIENT_ID</code>	OAuth2 Client ID Google
<code>GOOGLE_CLIENT_SECRET</code>	OAuth2 Client Secret Google

11. Commandes utiles

```
# Créer un super-utilisateur (accès /admin/)
uv run python project/manage.py createsuperuser

# Générer de nouvelles migrations après modification des modèles
uv run python project/manage.py makemigrations

# Appliquer les migrations
uv run python project/manage.py migrate

# Lancer les tests
uv run python project/manage.py test app
```

12. Glossaire

Terme	Définition
API RESTful	Interface de programmation respectant les contraintes REST (stateless, ressources, HTTP verbs)
JWT	JSON Web Token — token signé permettant de transmettre des informations d'identité
Django Ninja	Librairie construite sur Django permettant de créer des APIs REST typées avec Pydantic
ORM	Object-Relational Mapping — abstraction qui permet de manipuler la BDD via des objets Python
Transaction atomique	Ensemble d'opérations BDD qui réussissent ou échouent ensemble
Race condition	Situation où deux processus concurrents modifient la même donnée, causant des incohérences
TCGDex	API publique fournissant les données des cartes Pokémon TCG